

Guida di Studio Completa: Progetto UNO Flip!

Questa guida è stata creata per aiutarti a ripassare tutto il codice del progetto, capirne l'architettura interna e prepararti al meglio per eventuali domande del professore durante l'esame.

1. Architettura Generale e Pattern di Design

Il progetto è strutturato seguendo una variante del pattern **MVC (Model-View-Controller)** e implementa lo **State Pattern** per la gestione delle schermate di gioco.

Pattern Chiave Utilizzati:

- 1. State Pattern (Gestione delle Scene):**
La classe astratta Scene definisce l'interfaccia comune per ogni schermata (handleInput, update, render). SceneManager funge da contesto della macchina a stati: possiede un puntatore unico alla scena corrente (`std::unique_ptr<Scene> currentScene`) e delega ad essa le chiamate ad ogni frame. Consente di cambiare schermata dinamicamente senza accoppiare le scene tra loro.
- 2. Separazione logica/grafica (MVC):**
La classe Partita non contiene alcun riferimento a SFML, sprite o finestre. Gestisce unicamente lo stato della partita, le regole del gioco e i calcoli logici.
- 3. GameScene osserva lo stato di Partita e si occupa di disegnarlo a schermo tramite SFML 3, inviando gli input dell'utente alla classe di logica.**

2. La Logica di Gioco (Modello)

[Carta.h](#) e [carta.cpp](#)

Rappresenta una carta a **doppia faccia**: * **Lato Chiaro (Light Side)**: Colori classici (ROSSO, GIALLO, VERDE, BLU) e valori/effetti standard (SALTA, INVERTI, FLIP, PESCA_UNO, JOLLY, JOLLY_PESCA_DUE). * **Lato Oscuro (Dark Side)**: Colori alternativi (ROSA, VERDE_ACQUA, ARANCIONE, VIOLA) ed effetti più severi (SALTA_TUTTI, PESCA_CINQUE, JOLLY_PESCA_COLORE, più i classici INVERTI e FLIP). * La classe memorizza permanentemente entrambi i lati (`coloreChiaro/valoreChiaro` e `coloreOscuro/valoreOscuro`). Quando la partita esegue un "FLIP", le carte in mano e nel mazzo non cambiano istanza: cambia solo il booleano globale `latoOscuroAttivo` che determina quale faccia viene letta tramite i metodi `getColore()` e `getValore()`.

[Mazzo.h](#) e [Mazzo.cpp](#)

Gestisce le 112 carte del gioco, divise in un mazzo coperto (`carteDaPescare`) e una pila scoperta (`carteScartate`). * **Determinismo Cross-Platform (Molto importante per l'esame!)**: * In multiplayer, se un client gira su Windows (MSVC) e l'altro su Mac (Clang), l'uso di generatori casuali standard della libreria C++ (come `std::default_random_engine` o `rand()`) e `std::shuffle` può produrre accoppiamenti e mescolamenti diversi anche partendo dallo stesso seed, a causa delle differenze nelle implementazioni della libreria standard. * Per risolvere questo problema, è stato implementato un generatore custom basato su un **Generatore Congruenziale Lineare (LCG)**:
$$\text{seed} = (\text{seed} \times 1664525 + 1013904223) \bmod 2^{32}$$
 * Questo algoritmo è completamente deterministico e identico su qualunque macchina. Viene sincronizzato scambiando il seed iniziale in rete durante la lobby.

[Giocatore.h](#) e [Giocatore.cpp](#)

Modella il partecipante al tavolo: * Contiene il nome, la mano (vettore di Carta) e il flag `isBot`. * Se `isBot == true`, le mosse del giocatore vengono automatizzate dall'IA (definita in `Partita::mossaBot()`).

[Partita.h](#) e [Partita.cpp](#)

Contiene lo stato globale del gioco e la macchina a stati del turno: * **Turnazione**: Gestita da `turnoCorrente` (indice del vettore giocatori) e modificata da `sensoOrario` (che si inverte con la carta INVERTI). * **Regole di giocata (mossaValida)**: Controlla se la carta selezionata corrisponde al colore attivo, al valore attivo o se è una carta Jolly. * **Effetti delle carte (applicaEffettoCarta)**: * FLIP: Inverte il valore di `latoOscuroAttivo`, ricalcola il colore attivo basandosi sul lato opposto della carta in cima agli scarti e notifica il cambio. * SALTA_TUTTI: Salta tutti tranne chi ha

giocato la carta (in pratica regala un altro turno consecutivo in una partita a più giocatori). * JOLLY_PESCA_COLORE: Chi gioca sceglie un colore. Il giocatore successivo deve pescare carte finché non ne trova una di quel colore specifico.

3. L'Interfaccia Grafica (SFML 3)

Il Loop Principale ([Main_grafico.cpp](#))

Crea la finestra di SFML e ospita il ciclo di gioco principale. Calcola il `deltaTime` ad ogni iterazione per aggiornare le scene in modo indipendente dal frame rate.

Gestione delle Scene

- [LoginScene.cpp](#): Gestisce l'inserimento testuale del nome utente.
- [MenuScene.cpp](#): Schermata principale con pulsanti interattivi.
- [LobbyScene.cpp](#): Setup di rete. Il server si mette in ascolto non bloccante mostrando il proprio IP locale. Il client inserisce l'IP per connettersi.
- [GameScene.cpp](#): La scena più complessa.
- Disegna il tavolo da gioco, la carta in cima agli scarti e le carte della mano del giocatore in basso (calcolando le posizioni orizzontali in base al numero di carte per non farle uscire dallo schermo).
- Mostra le carte degli avversari coperte (o mostra i loro nomi e quante carte hanno in mano).
- Gestisce pulsanti per: Pescare, Dichiarare "UNO!", Contestare la mancata dichiarazione di "UNO!", Salvare la partita, ed effettuare il Rage Quit (abbandono).

Risorse e Utility

- [UIUtils.h](#): Fornisce metodi helper per disegnare pulsanti animati (con effetto hover), caselle di testo per l'input e finestre di dialogo personalizzate (es. la scelta del colore per le carte Jolly).
- [SoundManager.h](#): Gestore dell'audio che precarica i file audio (.wav) in memoria ed esegue i suoni di gioco (giocata, pesca, errore, vittoria, dichiarazione di UNO) per rendere l'esperienza premium.

4. Il Sistema Multiplayer (Rete)

Il multiplayer è peer-to-peer (1 Server + 1 Client) e fa uso di `sf::TcpSocket` in modalità **non bloccante**.

[GestoreRete.h](#) e [GestoreRete.cpp](#)

- Per default, i socket TCP di SFML sono bloccanti (ovvero il programma si congela in attesa di ricevere dati dalla rete).
- Nel nostro progetto sono configurati come **non bloccanti** (`socket.setBlocking(false)`). Questo permette al ciclo update della grafica di continuare a girare fluidamente a 60+ FPS anche se non ci sono pacchetti in arrivo.
- Il metodo `riceviMessaggio()` tenta di leggere dal socket; se non ci sono dati, ritorna immediatamente una stringa vuota invece di fermare il gioco.

Flusso di Sincronizzazione in Rete

1. **Lobby**:
2. Il Server genera un seed di rete casuale (es. `Mazzo::ottieniNumeroCasuale()`).
3. Invia un pacchetto iniziale al Client con il seed: `SEED; [valore_seed]`.
4. Entrambi impostano il seed nel loro Mazzo. Da questo momento in poi, le pescate e i mescolamenti avverranno in perfetto sincrono senza bisogno di scambiarsi l'intero mazzo a ogni mossa.
5. **Durante il Gioco (GameScene)**:
6. I giocatori si scambiano stringhe di testo codificate contenenti l'azione eseguita. Ad esempio:
 - `MOSSA; [indice_carta]; [colore_scelto]; [ha_detto_uno]` (quando un giocatore gioca una carta).
 - `PESCA` (quando il giocatore avversario pesca dal mazzo).
 - `CONTESTAZIONE` (se l'avversario contesta il mancato click su "UNO!").
 - `SALVATO; [nome_file]` (quando una partita viene salvata da uno dei due utenti, per sincronizzare il salvataggio).

sul disco dell'altro).

- `RAGE_QUIT` (se uno dei due chiude la partita o preme abbandona).

5. Persistenza dei Dati

[Database.h](#) e [Database.cpp](#)

Gestisce la classifica dei giocatori: * All'avvio, legge un file CSV (`classifica.csv`). Se non esiste, lo crea. * Parsa le righe spezzandole ad ogni virgola (formato: Nome, PartiteGiocate, Vittorie) e le inserisce in un vettore in RAM (records). * A fine partita incrementa le partite giocate di tutti i partecipanti e le vittorie del vincitore, ordinando la cache per numero di vittorie decrescenti. * Scrive nuovamente i record su disco sovrascrivendo il CSV.

Salvataggio Partita (`Partita::salvaPartita`)

Consente di salvare lo stato del gioco e riprenderlo in seguito: * Scrive in un file di testo (`salvataggio.txt`) tutte le variabili fondamentali: * Il valore di `latoOscuroAttivo`, `turnoCorrente`, `sensoOrario` e `coloreAttivo`. * Le carte attualmente presenti negli scarti. * Il mazzo delle carte da pescare rimaste. * I giocatori (nomi, bot e le carte che hanno attualmente in mano). * Quando si sceglie "Carica Partita" dal menu, `Partita::caricaPartita` effettua il parsing del file di testo e ricostruisce l'esatto stato della partita. In multiplayer, questa operazione viene sincronizzata affinché entrambi i computer scrivano e carichino lo stesso identico stato.

6. Domande Tipiche all'Esame (e come rispondere)

Domanda 1: "Come avete gestito il multiplayer e come evitate il lag nel ciclo grafico?"

- **Risposta:** "Abbiamo utilizzato la libreria SFML Network, in particolare i socket TCP. Per evitare che il gioco si blocchi in attesa di dati rallentando la grafica, abbiamo impostato i socket in modalità non bloccante (`setBlocking(false)`). In questo modo, ad ogni frame interroghiamo il socket per verificare la presenza di pacchetti; se non ce ne sono, il gioco continua a fare il rendering fluido delle animazioni."

Domanda 2: "Com'è possibile che il mazzo sia identico tra client e server senza inviare tutte le carte in rete ogni volta?"

- **Risposta:** "Per ridurre il traffico di rete ed evitare problemi di desincronizzazione tra sistemi operativi differenti (Mac e Windows utilizzano generatori casuali standard diversi), abbiamo implementato un generatore congruenziale lineare (LCG) proprietario in `Mazzo.cpp`. Durante la Lobby, il Server invia un singolo seed numerico al Client. Poiché l'algoritmo matematico di generazione dei numeri casuali è identico, da quel momento in poi sia il client che il server generano la stessa sequenza di accoppiamenti delle carte e di mescolamenti in modo deterministico e locale."

Domanda 3: "Come avete strutturato il codice per non mischiare logica e grafica?"

- **Risposta:** "Abbiamo seguito il pattern MVC. La classe `Partita` e le classi del modello (`Carta`, `Mazzo`, `Giocatore`) contengono pura logica C++ standard, senza dipendenze da SFML. Tutta la componente visuale, gli eventi di input della tastiera/mouse e i suoni sono incapsulati in `GameScene` e nelle altre classi derivate da `Scene`. `GameScene` legge lo stato logico di `Partita` e lo visualizza a schermo, fungendo da Controller."

Domanda 4: "Come funziona il passaggio da una schermata all'altra del gioco?"

- **Risposta:** "Abbiamo implementato uno State Pattern per le scene. C'è una classe base astratta `Scene` con metodi virtuali puri per input, update e rendering. La classe `SceneManager` mantiene un puntatore unico alla scena attiva (`std::unique_ptr<Scene> currentScene`). Quando un bottone richiede un cambio di schermata, ad esempio dal Menu alla Classifica, il `SceneManager` distrugge la vecchia scena e istanzia la nuova, garantendo una gestione efficiente della memoria."

Domanda 5: "Come gestite i salvataggi ed eventuali crash o abbandoni nel gioco?"

- **Risposta:** *"I salvataggi persistono lo stato completo di Partita (mazzo, scarti, mani dei giocatori, turno) in un file di testo (salvataggio.txt) strutturato in righe etichettate. La classifica invece usa un file CSV gestito dalla classe Database, che aggiorna le righe all'assegnazione della vittoria. Per gli abbandoni in multiplayer, abbiamo implementato un pacchetto di rete RAGE_QUIT: se un giocatore si disconnette o preme abbandona, l'altro riceve la notifica e vince a tavolino, aggiornando le statistiche."*